

CITY UNIVERSITY MALAYSIA
Faculty of Information Technology
Cyberjaya Campus

BIT1123 / BISE2093 / DIT1113

Object Oriented Programming

Assignment 1 — Individual (20%)

Self-Reflective Report

Student name	Hamza Gohar
Student ID	202504010004
Course	BIT1123 Object Oriented Programming
Lecturer	Sir Nazmirul Izzad Bin Nassir
GitHub	hamzagohar446-svg

This report looks back at the weekly Java tutorials and explains what I learned, where I got stuck, and how the exercises fit together.

1. Introduction

I started BIT1123 with only a basic idea of what a program looks like. I could follow a short example on the screen, but I was not yet comfortable writing a class of my own or explaining why the code was split into more than one file. The tutorials ran from a first Java program in Week 1 to a small quiz window in Week 10. This assignment collects that work in one repository and asks me to say, in my own words, what changed.

The early programs print text and read a few numbers. Later programs create objects, share code through a parent class, hide data behind methods, save a list to a file, and respond when a button is pressed. Writing the report made me go back through each folder and check that I could still explain the class I had put there.

2. Knowledge gained from each tutorial

Week 1 — first programs and decisions

Week 1 was about the shape of a Java file. HelloWorld.java has one class and a main method, and it prints my name and student ID. StudentGrade.java asks for a name, an ID, and three marks. I used a Scanner for the input and an if-else chain to turn the average into a letter grade. A mark of 50 or more is a pass. This week taught me that the program stops at the first condition that is true, so the order of the tests matters.

Week 2 — classes and objects

In Week 2 I stopped keeping every value inside main. Student.java holds the name, student ID, programme, and semester, and the constructor fills those fields when the object is created. displayProfile, attendClass, and submitWork are methods on that object. Main.java creates two students, Hamza Gohar and a second classmate, and calls the same methods on each one. Seeing two objects with the same methods but different data made the idea of a class much clearer than reading the definition alone.

Week 3 and 4 — inheritance and overriding

Person is the parent class. It stores a name and an ID and has an introduce method. Student and Lecturer extend Person, call the parent constructor, and replace introduce with their own message. Student can also study a topic, and Lecturer can conduct a class. In Main.java I put a visitor, a student, and a lecturer into one Person array. The loop calls introduce on each item. The output changes even though the call looks the same. That is the first time polymorphism felt concrete to me.

Week 5 — encapsulation

Week 5 uses a Student class again, but the fields are private. Main cannot write student.cgpa = 3.62. It has to call setCgpa. The setter refuses a CGPA below 0 or above 4, and it refuses a blank name or ID. I tested this by setting a valid CGPA of 3.62 and then trying 4.80. The printed CGPA stayed 3.62. documentation.txt in the same folder records that check. Extra methods felt slow at first. After the failed update I could see why the class should be the one that accepts or rejects a value.

Week 6 — protected fields and super

Employee stores a staff ID, a name, and a faculty with the protected modifier, so Lecturer can use them. The Lecturer constructor passes the shared values up with super and then stores the module and the room. displayTeachingLoad calls displayInfo from the parent and then prints the extra lines. I had mixed up the order before: the parent fields were still empty because super had not run yet. Putting super first fixed it.

Week 7 — abstraction

Appliance is abstract. turnOn and turnOff are written once. operate has no body in Appliance, so WashingMachine and Refrigerator must each write their own. The washing machine reports the load in kilograms. The refrigerator reports the target temperature. Main keeps both objects in an Appliance array and calls the same three methods. I cannot write new Appliance(...) because the class is not a finished machine. That restriction is what made the abstract keyword easier to remember.

Week 8 and 9 — ArrayList and files

The study notebook keeps Task objects in an ArrayList. A task has a description and a done flag. The menu can show the list, add a line, mark a number as done, save, or load. Saving writes each task as description|OPEN or description|DONE in tasks.txt. Loading reads those lines back into new Task objects. The file code sits in a try-catch because the file might be missing the first time the program runs. Starting the program from the week8-9 folder matters, because that is where tasks.txt is created.

Week 10 — Swing and events

CampusQuiz opens a window instead of printing to the console. Question stores the prompt, four options, and the index of the right answer. QuestionBank fills a list of eight questions about the topics from the earlier weeks. The window uses radio buttons, a Check answer button, a Next button, and a Restart button. CampusQuiz implements ActionListener, so actionPerformed runs when a button is pressed. A correct choice adds one to the score. After the last question a dialog shows the total. The layout took several tries before the question, the options, and the feedback area all stayed visible.

3. Challenges encountered

The first difficulty was simply getting a file to compile. A missing semicolon or a capital letter in the wrong place stopped the whole program, and the error message pointed at a line I did not always understand. Once classes started, I also confused the class with the object. I would change a field on one student and expect the other student to change as well.

Inheritance was the next hard part. I knew a subclass could add methods, but I did not see why the parent constructor had to run, or why introduce in Student hid the version in Person. Encapsulation felt like extra typing until I watched an invalid CGPA get rejected.

File handling and the graphical quiz were the two topics I had to repeat the most. For the file, the program threw an error when tasks.txt was not in the folder I launched from. For the quiz, the window opened but the question text and the buttons overlapped until I separated the header, the options, and the feedback into their own panels. I also found GitHub unfamiliar: creating one repository, keeping a single naming style for the week folders, and putting the report in the root all had to be done on purpose.

4. How the challenges were overcome

I worked one week at a time and ran the program after each small change. When a compile error appeared, I read the line number and fixed that line before adding anything new. For classes and objects I drew the two students on paper and wrote the field values beside each name. That stopped me from treating the class as if it were one shared record.

For inheritance I printed the result of the Person array in Week 3–4 and compared it with a direct call on the student. The two outputs showed me that the method which runs depends on the real object, not only on the type of the variable. For Week 6 I checked that super was the first statement in the Lecturer constructor.

For the notebook I saved one task, opened tasks.txt in a text editor, and then used the reload option to see the same line come back. For the quiz I added one button and one listener first, confirmed that a click changed the feedback text, and only then added the score and the next-question step. I also kept every tutorial in its own folder so a later week could not overwrite an earlier class with the same name.

5. Improvements in my Java programming skills

At the start of the course I needed an example in front of me to write a main method. By Week 10 I can create a class, give it a constructor, and call it from another file without copying the whole program into one place. I am more careful with types: a mark is a double, a semester is an int, and a done flag is a boolean.

I can also read a short Java file and say what each class is for. In the quiz, Question holds one item, QuestionBank holds the list, and CampusQuiz draws the window. Splitting the work that way is something I would not have done in Week 1. I still write slowly when the code uses files or Swing, but I can find the method that failed instead of rewriting the whole file.

6. Understanding of object-oriented programming

Object-oriented programming, as I understand it from these tutorials, means the program is organised around objects and the classes that describe them. The ideas below are the ones I can now point to in my own folders.

- **Class and object.** Student in Week 2 is the blueprint. The two variables in Main are the objects.
- **Encapsulation.** Week 5 keeps the fields private and checks the CGPA before it is stored.
- **Inheritance.** Student and Lecturer in Week 3–4, and Lecturer in Week 6, reuse a parent class.
- **Polymorphism.** A Person variable and an Appliance variable can refer to different subclasses, and the matching method runs.
- **Abstraction.** Appliance in Week 7 states that every machine can operate, and each subclass says how.

These are not separate topics that end when the week ends. The quiz in Week 10 is easier to change because the questions are objects in a list, which is the same ArrayList idea as the task notebook. The score is updated inside the window class rather than from outside, which is the same reason the CGPA setter exists.

7. Future learning plans

I want to keep writing small Java programs outside the weekly sheets, especially programs that save data and programs with more than one screen. I am still slow with exceptions and with arranging Swing panels, so those are the two areas I will practise first. I also want to use Git for every exercise, not only for this assignment, so that I can see what changed when something stops compiling.

After I am steady with these fundamentals I would like to build a slightly larger campus tool, such as a subject registration list that uses classes, a file, and a simple window together. That would show me whether I can choose the class boundaries myself instead of following a weekly topic.

8. Conclusion

This assignment gathered ten weeks of Java work into one repository and made me explain it. I began with a program that prints a line. I finished with classes that inherit, hide their data, save a list, and react to a button. The hardest weeks were the file notebook and the quiz window. Checking the saved file, and testing one button before adding the rest, is what got those two working.

I do not think I am finished with Java. I do think I can now open one of these folders and say what the class is responsible for, and I can add a small change without starting again. That is the progress I want to take into the next part of the course.

9. GitHub repository

All source files for Week 1 to Week 10, this report, and the README are kept in a single repository under the account hamzagohar446-svg. The public URL for submission is:

<https://github.com/hamzagohar446-svg/BIT1123-OOP-Assignment1>

The same address is written in README.md. Student name: Hamza Gohar. Student ID: 202504010004.